Proof-of-Concept Kit — Microsoft Defender for Identity

Implementation Solution Guide

Version 1.0.0 · 2026-07-11

Contents

Executive Summary — Proof-of-Concept Kit: Microsoft Defender for Identity 5

What you get . . . 5

What this delivers . . . 5

The standard deployment model . . . 5

Boundary . . . 5

Effort . . . 6

1. Solution Overview — Proof-of-Concept Kit: Microsoft Defender for Identity 6

What Microsoft Defender for Identity is . . . 6

What a Proof-of-Concept Kit is . . . 6

Why this matters . . . 7

How it works . . . 7

What you can control . . . 7

Limitations (be honest with buyers) . . . 7

Connections and authentication used . . . 8

The standard deployment model . . . 8

2. Requirements Guide — Proof-of-Concept Kit: Microsoft Defender for Identity 8

2.1 Defender for Identity plan available to activate . . . 8

2.2 A pilot domain controller for the sensor . . . 9

2.3 Directory Service Account (DSA) . . . 9

2.4 Network reach from the pilot DC to the MDI service . . . 9

2.5 A test workstation and test account for benign recon . . . 9

2.6 A honeytoken decoy account . . . 9

2.7 Evaluation criteria agreed up front . . . 9

2.8 Advanced-hunting read access . . . 10

Checklist . . . 10

Global Requirements — Defender (category-level) 10

G1. Microsoft 365 / Entra tenant . . . 10

G2. Defender licensing . . . 10

G3. Administrative roles (least privilege) . . . 11

G4. Onboarding reach to endpoints / identity / workloads . . . 11

G5. Network access to Defender service endpoints . . . 11

G6. A pilot ring . . . 11

G7. Log Analytics workspace (where required) . . . 11

G8. Response governance . . . 11

3. Implementation Guide — Proof-of-Concept Kit: Microsoft Defender for Identity 12

Naming (set once, reuse verbatim) . . . 12
Step 1 — Assess readiness (read-only) . . . 12
Step 2 — Enable MDI and install the pilot sensor . . . 12
Step 3 — Generate benign test signal . . . 13
Step 4 — Validate and demonstrate the detections . . . 13
Step 5 — Score against your criteria . . . 13
Step 6 — Tear down (clean rollback) . . . 13
Step 7 — Decide: expand or stop . . . 14
5. Testing Guide — Proof-of-Concept Kit: Microsoft Defender for Identity 14
Test 1 — Pilot sensor healthy . . . 14
Test 2 — Honeytoken tagged . . . 14
Test 3 — Benign recon signal generated . . . 14
Test 4 — Reconnaissance alert reached the portal . . . 14
Test 5 — Honeytoken activity flagged . . . 15
Test 6 — Identity timeline & hunting visibility . . . 15
Test 7 — Scorecard produced . . . 15
Test 8 — Clean teardown . . . 15
6. Troubleshooting Guide — Proof-of-Concept Kit: Microsoft Defender for Identity 15
Sensor unhealthy . . . 15
Honeytoken not flagged . . . 16
Recon signal not generated . . . 16
No reconnaissance alert . . . 16
Hunting tables empty . . . 16
DSA problems . . . 16
Script cannot authenticate . . . 16
Teardown left residue . . . 16
7. Best Practices — Proof-of-Concept Kit: Microsoft Defender for Identity 17
Keep the PoC scoped and reversible . . . 17
Benign signal only — never real attacks . . . 17
Use a honeytoken for a clean, high-fidelity signal . . . 17
Respect the domain controller . . . 17
Set the bar before you run . . . 17
Allow warm-up before judging . . . 17
Keep response approval-gated . . . 17
Tear down completely . . . 18
Keep licensing and preview facts current . . . 18

Where to go next 18
8. Deliverables Summary — Proof-of-Concept Kit: Microsoft Defender for Identity 18
Documentation 18
PowerShell scripts (Microsoft Graph Security + directory enumeration + KQL) 19
What the buyer receives 19
Outcome 19
Appendix A — PowerShell Scripts 20

Executive Summary — Proof-of-Concept Kit: Microsoft Defender for Identity

White-label · Defender

Deciding on Microsoft Defender for Identity (MDI) shouldn't rest on a slide deck. This Proof-of-Concept (PoC) Kit stands up a **scoped, safe** proof of concept in your own tenant: it enables MDI on a single **pilot domain controller**, generates **benign, simulated** test signal (harmless read-only directory enumeration and a Microsoft **honeytoken** decoy account — never a real attack, exploit, or credential theft), demonstrates the headline **identity threat detection & response (ITDR)** capabilities on that signal, scores the result against your evaluation criteria, and then **cleanly tears the PoC back down**.

The kit is written for an administrator with no prior Defender experience. Everything is **pilot-scoped and reversible**: you deploy one sensor, prove the value, capture a scorecard, and uninstall — with the rest of Active Directory untouched.

What you get

- A plain-language **Solution Overview** of MDI, ITDR, and how a PoC proves it on real-but-safe signal.
- A **Requirements Guide** of the PoC-specific prerequisites (the category-wide ones live in `global-requirements-defender.md`).
- An **Implementation Guide** — exact, ordered steps to deploy a pilot sensor, generate benign signal, demonstrate the detections, score, and tear down.
- Five **PowerShell scripts** (Microsoft Graph Security + directory enumeration) with **KQL advanced-hunting** queries: a `-WhatIf`-gated pilot enable/honeytoken helper, a **benign** recon test-signal generator, a demo/validate check, a criteria scorecard, and a `-WhatIf`-gated teardown.
- **Testing**, **Troubleshooting**, and **Best Practices** guides.

What this delivers

A concrete, **in-tenant proof** that Microsoft Defender for Identity delivers ITDR — a pilot sensor raising a real detection from benign reconnaissance and honeytoken activity, the alert visible in the incident queue and on the **identity timeline**, the compromised-path and **UEBA** views populated, and a **scorecard** mapping each observed capability to your pass/fail criteria — all torn down cleanly at the end.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — here scoped as a PoC: assess readiness, deploy one pilot sensor, generate benign signal, validate and score, then either expand or tear down.

Boundary

This is an **evaluation kit**, not a production rollout. It proves MDI on one pilot DC and then removes the sensor. Fleet-wide sensor deployment, AD hardening, and long-term operations are separate adjacent solutions (see *Where to go next*). All test signal is **benign and simulated** — read-only enumeration and a decoy account. Response actions are kept **approval-gated**; incident-response decisions remain the

SOC/security team's.

Effort

A competent administrator can complete the full PoC — deploy, signal, validate, score, and teardown — in a short, focused engagement on one domain controller in an approved change window. Sensor install requires a maintenance window on the pilot DC; everything else is reversible.

1. Solution Overview — Proof-of-Concept Kit: Microsoft Defender for Identity

What Microsoft Defender for Identity is

Microsoft Defender for Identity (MDI) is Microsoft's **identity threat detection & response (ITDR)** platform for on-premises Active Directory (and AD FS / AD CS / Entra Connect where present). Lightweight **sensors** installed directly on domain controllers watch AD traffic and behaviour and stream detections to the unified Defender portal at `security.microsoft.com`. Its headline capabilities are:

- **Reconnaissance detection** — enumeration of users, groups, and machines (LDAP/SAMR/DNS recon), the classic first step of an intrusion.
- **Lateral-movement & credential-attack detection** — Pass-the-Hash/Ticket, overpass-the-hash, DCSync, and suspicious authentication patterns.
- **Identity timeline & investigation** — a per-identity activity timeline stitched into Defender XDR incidents.
- **User & entity behavioural analytics (UEBA)** and **investigation-priority scoring**.
- **Honeytoken accounts** — decoy identities that should never be used, so *any* activity against them is a high-fidelity signal.

What a Proof-of-Concept Kit is

A PoC Kit is a **customer-runnable evaluation**, not a production deployment and not a sales demo. Its job is to answer one question — *does MDI actually detect identity attacks against our AD?* — with evidence, safely, in your own tenant. It does that by:

1. **Enabling** MDI with a single **sensor on a pilot domain controller**.
2. **Generating benign test signal** — harmless, **read-only directory enumeration** (the same `net group` / `nltest` / LDAP queries an admin runs every day) to trigger reconnaissance detections, plus a Microsoft **honeytoken** decoy account whose use is a clean, high-fidelity trigger. **No real attack, exploit, credential theft, or malware is ever used.**
3. **Demonstrating** that the signal produces a real detection — an alert in the incident queue, an entry on the identity timeline, rows in advanced hunting, and UEBA scoring.
4. **Scoring** what you observed against your evaluation criteria.
5. **Tearing down** — uninstalling the pilot sensor and removing everything the PoC created.

Why this matters

Identity is the modern battleground, and most breaches move *through* Active Directory. A vendor deck can't show you how MDI behaves against *your* DCs, *your* directory, and *your* admin patterns. A scoped PoC replaces "trust the deck" with "watch it catch the recon." Because it uses **one pilot sensor and benign signal**, the rest of AD is untouched and you get proof without a fleet-wide deployment before you've seen the value.

How it works

1. **Assess** readiness with `POCMDI-Enable.ps1 -WhatIf` — MDI plan, admin roles, a Directory Service Account (DSA), a candidate pilot DC, and network reach.

2. **Configure & enable** — activate MDI, register the DSA `MDI PoC – DSA – Pilot`, and install the sensor `MDI PoC Sensor – Pilot DC – Pilot` on one domain controller in a change window. Tag a **honeytoken** decoy account `MDI PoC Honeytoken – Pilot`.

3. **Generate benign signal** with `POCMDI-TestSignal.ps1` — run harmless read-only enumeration from a test workstation and touch the honeytoken account.

4. **Validate & demonstrate** with `POCMDI-Validate.ps1` — confirm the reconnaissance and honeytoken alerts reached the portal, the identity timeline, and hunting (`IdentityQueryEvents`, `IdentityLogonEvents`, `IdentityDirectoryEvents`, `AlertInfo`).

5. **Score** with `POCMDI-Scorecard.ps1` against your criteria.

6. **Tear down** with `POCMDI-Teardown.ps1 -WhatIf` first, then for real — untag the honeytoken, uninstall the sensor, remove the DSA, resolve test alerts.

What you can control

• **Pilot sensor** — one domain controller only; the rest of the fleet is untouched.

• **Which capabilities to prove** — recon detection, honeytoken, identity timeline, UEBA, XDR correlation.

• **Response posture** — any response kept in an **approval/simulate** posture throughout the PoC.

• **Scorecard criteria** — the pass/fail bar you evaluate against.

• **Teardown** — full, reversible removal.

Limitations (be honest with buyers)

• MDI is **license-gated** (Defender for Identity, often bundled in E5 / E5 Security). *Verify current plan availability in your tenant.*

• The **sensor runs on a domain controller** and requires resources, network reach, and a change window — it is not a portal-only toggle, and it cannot be installed agentlessly.

• Test signal is **benign and simulated only** — read-only enumeration and a decoy account. This proves the detection pipeline, not resilience to a real adversary (that is a red-team engagement, out of scope).

• Advanced-hunting tables and UEBA scoring **populate over time** — allow warm-up after sensor install before judging.

• This is a **PoC**, not a production rollout — it stays on one DC and is designed to be removed. Fleet-wide deployment is a separate solution.

• MDI improves detection; it does not replace AD hardening, tiered admin, or the SOC's incident-response decisions.

Connections and authentication used

• The **unified Defender portal** (`security.microsoft.com`) for MDI activation, sensor management, honeytoken configuration, incidents, and the identity timeline.

• **Microsoft Graph Security** and the advanced-hunting API (`IdentityQueryEvents`, `IdentityLogonEvents`, `IdentityDirectoryEvents`, `AlertInfo`) for read-only validation and scoring.

• **Directory enumeration** (LDAP / `net` / `nltest`) from a test workstation for the benign recon signal.

• Least-privilege roles: **Security Administrator** for configuration, **Security Reader** for read-only work, **Domain Admin** (change window) for the one-time sensor install, and **Global Administrator** only for one-time first-service activation.

The standard deployment model

> ***Assess readiness → configure/activate MDI and install one pilot sensor → generate benign recon + honeytoken signal → validate the detections and score → expand or tear down.***
> *Repeatable pattern: **Assess → Configure → Pilot → Validate → Expand**, run here as a safe, reversible PoC with response kept approval-gated.*

2. Requirements Guide — Proof-of-Concept Kit: Microsoft Defender for Identity

Verify **every** requirement before enabling anything. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites (tenant, licensing, admin roles, onboarding reach, network access, pilot ring, workspace, response governance) are in `global-requirements-defender.md` — do not repeat them; the items below are **specific to running a scoped MDI proof of concept**. Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, and test.

2.1 Defender for Identity plan available to activate

Description: A licensed **Defender for Identity** plan the tenant can activate for the PoC. **Why:** ITDR sensors, detections, and the identity timeline are the capabilities being proven; without the plan MDI cannot be activated. **Verify:** Admin center → **Billing** → **Licenses** shows Defender for Identity; `security.microsoft.com` → **Settings** → **Identities** loads. Run `POCMDI-Enable.ps1 -WhatIf`. *Verify current plan availability in your tenant.* **Implement-if-missing:** Assign the Defender for Identity plan (often bundled in M365 E5 / E5 Security) before enabling.

2.2 A pilot domain controller for the sensor

Description: One **domain controller** to host the pilot sensor `MDI PoC Sensor – Pilot DC – Pilot`, with a supported OS, adequate resources, and a **change window** to install. **Why:** MDI detects on-prem identity attacks from a sensor installed directly on a DC; the PoC deliberately uses one DC so it can be uninstalled cleanly. **Verify:** `POCMDI-Enable.ps1 -WhatIf` lists candidate DCs; confirm OS support, resources, and a maintenance window. *Verify current sensor OS support and sizing.* **Implement-if-missing:** Choose a non-critical DC (or a lab DC), confirm prerequisites, and schedule the change window.

2.3 Directory Service Account (DSA)

Description: A read-only **Directory Service Account** for MDI (gMSA recommended), named `MDI PoC – DSA – Pilot`. **Why:** The sensor uses the DSA to query AD for additional context; without it, some detections are limited. **Verify:** Confirm a gMSA (or standard read-only account) is created; `POCMDI-Enable.ps1 -WhatIf` notes DSA readiness. *Verify current MDI DSA requirements.* **Implement-if-missing:** Create the gMSA and register it under **Settings** → **Identities** → **Microsoft Defender for Identity** → **Directory Service accounts**.

2.4 Network reach from the pilot DC to the MDI service

Description: Outbound HTTPS from the pilot DC to the Microsoft Defender for Identity service URLs. **Why:** The sensor reports to the cloud service; blocked endpoints break sensor health and detections. **Verify:** Confirm the documented MDI service URLs are allowed from the pilot DC. *Verify current endpoint list.* **Implement-if-missing:** Allow the MDI service endpoints through proxy/firewall for the pilot DC.

2.5 A test workstation and test account for benign recon

Description: A non-privileged **test workstation** and a **test account** from which to run harmless read-only directory enumeration. **Why:** The benign recon signal (LDAP/`net`/`nltest` enumeration) must originate from a real client so the sensor observes it; using a dedicated test account keeps the signal attributable and the teardown clean. **Verify:** You can sign in to a domain-joined test workstation with a test account. **Implement-if-missing:** Provision a test workstation and a low-privilege test account before Step 3.

2.6 A honeytoken decoy account

Description: A dormant, unused **decoy account** to tag as a honeytoken (`MDI PoC Honeytoken – Pilot`). **Why:** A honeytoken should never be used legitimately, so any activity against it is a clean, high-fidelity detection — ideal for a PoC. **Verify:** An unused AD account exists to tag; `POCMDI-Enable.ps1 -WhatIf` reports honeytoken status. **Implement-if-missing:** Create (or pick) a dormant account and plan to tag it as a honeytoken in MDI settings. *Do not tag a real, in-use account.*

2.7 Evaluation criteria agreed up front

Description: A short, written list of what "pass" means — e.g. *the reconnaissance alert fires within N minutes, appears on the identity timeline, is queryable in hunting, and the honeytoken activity is flagged.* **Why:** A PoC is only conclusive if the bar is set **before** you run it; the scorecard maps observations to these criteria. **Verify:** A criteria list exists; `POCMDI-Scorecard.ps1` accepts it as input. **Implement-if-missing:**

Draft 5–8 criteria with the evaluating stakeholders before Step 1. *What counts as "good enough" is the customer's decision.*

2.8 Advanced-hunting read access

Description: Permission to run advanced-hunting queries via the portal and the Graph Security API (`ThreatHunting.Read.All`). **Why:** The validate and scorecard scripts confirm the signal landed in `IdentityQueryEvents`, `IdentityLogonEvents`, `IdentityDirectoryEvents`, and `AlertInfo`. **Verify:** `security.microsoft.com` → **Hunting** → **Advanced hunting** runs a query; `POCMDI-Validate.ps1` returns rows. **Implement-if-missing:** Grant the running account **Security Reader** (or higher) and consent to the read-only hunting scope.

Checklist

- Defender for Identity plan available to activate — `POCMDI-Enable.ps1 -WhatIf`
- A pilot domain controller + change window for `MDI PoC Sensor – Pilot DC – Pilot`
- Directory Service Account `MDI PoC – DSA – Pilot` (gMSA)
- Network reach from the pilot DC to the MDI service URLs
- A test workstation and test account for benign recon
- A dormant honeytoken decoy account `MDI PoC Honeytoken – Pilot`
- Evaluation criteria agreed and written down — `POCMDI-Scorecard.ps1`
- Advanced-hunting read access (`ThreatHunting.Read.All`)

Global Requirements — Defender (category-level)

These requirements apply to every solution in the Defender category (Microsoft Defender XDR and Defender for Cloud). Published once at the category level so listings stay small. Confirm before deploying any Defender solution. Each entry: Description / Why / Verify / Implement-if-missing.

> *Defender changes touch detection and response for the whole tenant. Onboard in a ring (pilot devices/ users first), keep automated response in an approval/simulate posture until trusted, and never disable an existing control to "test."*

G1. Microsoft 365 / Entra tenant

Description: An active Microsoft 365 tenant with Entra ID. Why: Defender XDR services and the unified portal (security.microsoft.com) are tenant workloads. Verify: Sign in to `https://security.microsoft.com` as an administrator. Implement: Provision a Microsoft 365 tenant and confirm Entra ID identities exist.

G2. Defender licensing

Description: The Defender plans for the pillars in scope — e.g. Defender for Endpoint P1/P2, Defender for Identity, Defender for Office 365 P1/P2, Defender for Cloud Apps, and Defender for Cloud plans. Why:

Each pillar/feature is license-gated; onboarding without the plan fails or silently limits features. Verify: Admin center -> `Billing -> Licenses` (or the Defender portal). *Confirm the exact plan each feature needs — verify current availability in your tenant.* Implement: Assign the required plans (often bundled in M365 E5 / E5 Security) before onboarding.

G3. Administrative roles (least privilege)

Description: Security Administrator (and Global Administrator for first-time service activation). Why: Onboarding, RBAC, and policy changes in the Defender portal require these roles. Verify: Entra -> `Roles and administrators`; confirm Security Administrator. Implement: Assign Security Administrator; reserve Global Administrator for one-time activations.

G4. Onboarding reach to endpoints / identity / workloads

Description: A management channel to onboard the pillars — Intune/Configuration Manager/GPO/script for endpoints, domain controllers/AD FS for Identity sensors, and cloud connectors for Defender for Cloud. Why: Each pillar collects signal from a source that must be reachable and instrumented. Verify: Endpoints are managed (Intune/CM); DCs are reachable; cloud subscriptions are connected. Run the solution's discovery script. Implement: Establish the onboarding channel per pillar (see the relevant catalog endpoint/identity solutions).

G5. Network access to Defender service endpoints

Description: Outbound HTTPS to the Microsoft Defender / Microsoft 365 service URLs from onboarded assets. Why: Sensors and agents report to the cloud service; blocked endpoints break onboarding and detections. Verify: Confirm the documented Defender endpoint URLs/IP ranges are allowed. *Verify current endpoint list.* Implement: Allow the Defender/M365 endpoints through proxy/firewall.

G6. A pilot ring

Description: A defined set of pilot devices/users/subscriptions to onboard first. Why: XDR onboarding and automated response are validated on a ring before tenant-wide rollout. Verify: A pilot group exists in Entra/Intune. Implement: Create the pilot ring before enabling anything that acts automatically.

G7. Log Analytics workspace (where required)

Description: A Log Analytics workspace for Defender for Cloud / advanced hunting export where used. Why: Some Defender data and continuous export land in a workspace. Verify: A workspace exists and is access-controlled. Run the discovery script. Implement: Provision a workspace (or reuse the SIEM/observability one) with least-privilege access.

G8. Response governance

Description: An agreed position on automated response (auto-remediation levels, approval requirements). Why: XDR/AIR can take disruptive actions (isolate, delete, disable); the organization must decide the guardrails. Verify: A documented response policy exists; automation is in an approval/simulate posture until trusted. Implement: Define response guardrails; keep AIR/automation approval-gated at first. *Incident-response decisions are the SOC/security team's; this guide covers the technical configuration.*

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation Guide — Proof-of-Concept Kit: Microsoft Defender for Identity

Follow these steps **in order**. Configuration is in the **unified Defender portal** (`https://security.microsoft.com` → **Settings** → **Identities**); the sensor installs on the pilot **domain controller** in a change window. Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, and test.

> ***Model (run as a PoC):*** *Assess readiness, install **one** sensor on a pilot DC, generate **benign** test signal, validate the detections and score them, then **tear down**. All test signal is harmless read-only enumeration and a honeytoken decoy — **never a real attack, exploit, or credential theft**. The rest of AD is untouched.*

Naming (set once, reuse verbatim)

- Pilot sensor / DC: `MDI PoC Sensor – Pilot DC – Pilot`
- Directory Service Account: `MDI PoC – DSA – Pilot`
- Honeytoken decoy account: `MDI PoC Honeytoken – Pilot`
- Scorecard output file: `MDI PoC Scorecard – Pilot DC – Validate`
- Test-signal tag (used in notes/hunting): `MDI-PoC-TESTSIGNAL`

Step 1 — Assess readiness (read-only)

1. Sign in to `security.microsoft.com` as a **Security Administrator**.
2. Run `POCMDI-Enable.ps1 -WhatIf` — previews (changes nothing): the MDI plan, admin roles, whether MDI is activated, candidate DCs, the DSA, and the honeytoken account.
3. Choose the pilot DC and confirm the change window, DSA (`MDI PoC – DSA – Pilot`), and a dormant honeytoken account (`MDI PoC Honeytoken – Pilot`).

Step 2 — Enable MDI and install the pilot sensor

1. `security.microsoft.com` → **Settings** → **Identities**. If MDI is not initialized, complete the one-time activation (Global Administrator, first time only).
2. **Settings** → **Identities** → **Directory Service accounts** → add `MDI PoC – DSA – Pilot` (gMSA recommended).
3. **Settings** → **Identities** → **Sensors** → **Add sensor** → download the sensor package and copy the **access key**.
4. In the approved change window, install the sensor on the pilot DC, supplying the access key. Name/label it `MDI PoC Sensor – Pilot DC – Pilot`.
5. **Settings** → **Identities** → **Sensors** — confirm the sensor shows **Running / Healthy**.

6. **Settings** → **Identities** → **Entity tags** → **Honeytoken** → tag `MDI PoC Honeytoken – Pilot`. (Or run `POCMDI-Enable.ps1` without `-WhatIf` — it is `-WhatIf`-gated and can tag the honeytoken via Graph.)

Step 3 — Generate benign test signal

1. From the **test workstation**, signed in as the **test account**, run `POCMDI-TestSignal.ps1` (or its printed manual commands). It performs only **benign, read-only** actions tagged `MDI-PoC-TESTSIGNAL`:

• **Reconnaissance signal:** harmless directory enumeration — `net group "Domain Admins" /domain`, `net user /domain`, `nltest /domain_trusts`, and a read-only LDAP query for users/groups. These are the same commands admins run daily; MDI flags the *pattern* as reconnaissance.

• **Honeytoken signal:** a benign authentication or directory query touching `MDI PoC Honeytoken – Pilot` — because the decoy should never be used, any touch is a clean detection.

1. **Do not** run attack tools, credential-dumping utilities, exploitation frameworks, or anything that changes AD. Read-only enumeration and touching a decoy account are sufficient and safe.

2. Note the time of each signal for the scorecard latency comparison.

Step 4 — Validate and demonstrate the detections

1. Run `POCMDI-Validate.ps1` — it confirms (read-only): the **reconnaissance** and **honeytoken** alerts reached the portal (`AlertInfo`), the activity appears in `IdentityQueryEvents` / `IdentityLogonEvents` / `IdentityDirectoryEvents`, and the identity timeline populated.

2. In the portal, walk the demo: open the incident → the **alert story** → the **identity page** → **Timeline** → the **UEBA / investigation-priority** view. This is the headline-capability demo for stakeholders.

3. Confirm any response remains **approval-gated** (nothing auto-acted).

Step 5 — Score against your criteria

1. Run `POCMDI-Scorecard.ps1 -CriteriaPath .\criteria.json -OutFile "MDI PoC Scorecard – Pilot DC – Validate.json"`. It maps each observation (recon alert fired, honeytoken flagged, timeline present, hunting visible, UEBA scored) to your pass/fail criteria and writes the scorecard.

2. Review the scorecard with stakeholders — the decision artefact.

Step 6 — Tear down (clean rollback)

1. Run `POCMDI-Teardown.ps1 -WhatIf` first to preview, then for real.

2. Teardown: **untag** the honeytoken `MDI PoC Honeytoken – Pilot`, **uninstall** the sensor from the pilot DC (`security.microsoft.com` → **Settings** → **Identities** → **Sensors** → delete, then remove the sensor service on the DC), **remove** the DSA `MDI PoC – DSA – Pilot` if it was created only for the PoC, and **resolve** the `MDI-PoC-TESTSIGNAL` test alerts.

3. Keep the scorecard as your record.

Step 7 — Decide: expand or stop

1. If the PoC passed and you choose to proceed, move to a **fleet-wide MDI sensor deployment / AD hardening** engagement — a separate, adjacent solution (see *Where to go next*).
2. If not proceeding, confirm the sensor is uninstalled and healthy-count is zero with `POCMDI-Validate.ps1`.

Order of operations

`Assess (-WhatIf) → activate MDI + DSA + install one pilot sensor + tag honeytoken → generate benign recon/honeytoken signal → validate + demonstrate (timeline, UEBA) → score vs criteria → teardown (-WhatIf then real) → decide expand or stop`

5. Testing Guide — Proof-of-Concept Kit: Microsoft Defender for Identity

Run these checks during the PoC (Steps 2–6 of the Implementation Guide). Each test has a **pass/fail gate**. All signal is **benign** (read-only enumeration + a honeytoken decoy). Objects are referenced by the exact names from the Implementation Guide.

Test 1 — Pilot sensor healthy

Procedure: Run `POCMDI-Enable.ps1 -WhatIf`; after install, in **Settings** → **Identities** → **Sensors**, confirm `MDI PoC Sensor – Pilot DC – Pilot` shows **Running / Healthy**. **Pass:** The sensor is healthy and the DSA is configured. **Fail:** Sensor "Not configured / Unhealthy" — see Troubleshooting *Sensor unhealthy*.

Test 2 — Honeytoken tagged

Procedure: In **Settings** → **Identities** → **Entity tags** → **Honeytoken**, confirm `MDI PoC Honeytoken – Pilot` is tagged. **Pass:** The decoy account is tagged as a honeytoken. **Fail:** Not tagged — re-run Implementation Step 2.6; see Troubleshooting *Honeytoken not flagged*.

Test 3 — Benign recon signal generated

Procedure: From the test workstation/account, run `POCMDI-TestSignal.ps1`; confirm the enumeration commands and the honeytoken touch completed. **Pass:** The benign read-only enumeration ran and the honeytoken was touched. No attack tooling used. **Fail:** Commands blocked — confirm the test account can query AD; see Troubleshooting *Recon signal not generated*.

Test 4 — Reconnaissance alert reached the portal

Procedure: Run `POCMDI-Validate.ps1`. In **Incidents & alerts**, confirm a reconnaissance-type alert from the pilot activity. **Pass:** A real reconnaissance alert appears within your agreed latency window. **Fail:** No alert — see Troubleshooting *No reconnaissance alert*.

Test 5 — Honeytoken activity flagged

Procedure: Confirm the honeytoken touch raised an alert / is highlighted; `POCMDI-Validate.ps1` checks `AlertInfo` and identity events. **Pass:** Activity against `MDI PoC Honeytoken – Pilot` is flagged as high-fidelity. **Fail:** Not flagged — see Troubleshooting *Honeytoken not flagged*.

Test 6 — Identity timeline & hunting visibility

Procedure: Open the identity page → **Timeline**; `POCMDI-Validate.ps1` runs KQL over `IdentityQueryEvents`, `IdentityLogonEvents`, `IdentityDirectoryEvents`. **Pass:** The timeline shows the test activity and hunting tables return recent rows. **Fail:** Timeline/tables empty — allow warm-up; see Troubleshooting *Hunting tables empty*.

Test 7 — Scorecard produced

Procedure: Run `POCMDI-Scorecard.ps1` and confirm `MDI PoC Scorecard – Pilot DC – Validate.json` is written with a pass/fail per criterion. **Pass:** A complete scorecard maps every observation to your criteria. **Fail:** Missing observations — re-run the relevant validate step first.

Test 8 — Clean teardown

Procedure: Run `POCMDI-Teardown.ps1 -WhatIf`, then for real; re-run `POCMDI-Validate.ps1`. **Pass:** The honeytoken is untagged, the sensor uninstalled, the DSA removed (if PoC-only), and test alerts resolved. **Fail:** Residue remains — see Troubleshooting *Teardown left residue*.

Exit gate

Tests 1–7 pass to call the PoC conclusive; Test 8 passes to close it out cleanly. Keep the scorecard file as the decision record.

6. Troubleshooting Guide — Proof-of-Concept Kit: Microsoft Defender for Identity

Common field issues, in **symptom** → **cause** → **fix** form. Run `POCMDI-Enable.ps1 -WhatIf` and `POCMDI-Validate.ps1` first — most causes surface there. All test signal is benign (read-only enumeration + a honeytoken decoy).

Sensor unhealthy

Symptom: `MDI PoC Sensor – Pilot DC – Pilot` shows **Unhealthy** or **Not configured** in **Settings** → **Identities** → **Sensors**. **Cause:** Missing/mis-scoped DSA, blocked outbound reach to the MDI service, insufficient DC resources, or an unsupported sensor OS. **Fix:** Verify the DSA `MDI PoC – DSA – Pilot` under **Directory Service accounts**; confirm the DC can reach the MDI service URLs (per `global-requirements-defender.md` G5); confirm DC resources and a supported OS (*verify current sensor OS support*). Reinstall with a fresh access key if needed, then re-run `POCMDI-Validate.ps1`.

Honeytoken not flagged

Symptom: Test 5 fails — touching `MDI PoC Honeytoken – Pilot` did not raise anything. **Cause:** The account is not tagged as a honeytoken, the sensor is not yet healthy, or signal has not propagated. **Fix:** Confirm the tag under **Settings** → **Identities** → **Entity tags** → **Honeytoken**; confirm Test 1 passed; allow warm-up time and re-touch the account, then re-run `POCMDI-Validate.ps1`.

Recon signal not generated

Symptom: Test 3 fails — the enumeration commands are blocked or return nothing. **Cause:** The test account cannot query AD, or a security tool blocks the enumeration on the workstation. **Fix:** Confirm the test account can run `net group` / `nltest` / LDAP queries against the domain; run from a domain-joined test workstation; re-run `POCMDI-TestSignal.ps1`.

No reconnaissance alert

Symptom: Test 4 fails — no reconnaissance alert appears after the benign enumeration. **Cause:** The sensor is not healthy, the enumeration was too light to trip the detection threshold, or signal has not propagated. **Fix:** Confirm sensor health (Test 1); run the full enumeration set (the script runs several patterns); allow propagation time; re-run `POCMDI-Validate.ps1`. Some detections need a recognisable enumeration pattern, not a single query.

Hunting tables empty

Symptom: `POCMDI-Validate.ps1` returns no rows from `IdentityQueryEvents` / `IdentityLogonEvents` / `IdentityDirectoryEvents`. **Cause:** Tables populate over time after sensor install, or the account lacks hunting read scope. **Fix:** Allow warm-up (tables populate over minutes to hours); confirm `ThreatHunting.Read.All` consent and **Security Reader**; re-run the validate script.

DSA problems

Symptom: The sensor installs but detections are limited or the DSA test fails. **Cause:** The gMSA is not resolvable from the DC, lacks read permission, or is mis-registered. **Fix:** Confirm the gMSA `MDI PoC – DSA – Pilot` is installed and resolvable on the DC and registered under **Directory Service accounts**; *verify current MDI DSA requirements*.

Script cannot authenticate

Symptom: `POCMDI-*.ps1` fails on connect or returns access-denied. **Cause:** The Microsoft Graph modules are missing, read scopes were not consented, or the account lacks the role. **Fix:** Let the script install the modules (it checks on run); sign in with `Connect-MgGraph` and consent to the read-only scopes; confirm **Security Reader** (or Security Administrator for the honeytoken tag). Re-run.

Teardown left residue

Symptom: After `POCMDI-Teardown.ps1`, the sensor still appears or the honeytoken is still tagged. **Cause:** Teardown ran in `-WhatIf` only, or the sensor service on the DC was not removed. **Fix:** Run teardown **without** `-WhatIf`; delete the sensor in the portal **and** uninstall the sensor service on the DC;

untag the honeytoken; resolve the `MDI-PoC-TESTSIGNAL` alerts; re-confirm with `POCMDI-Validate.ps1`.

7. Best Practices — Proof-of-Concept Kit: Microsoft Defender for Identity

Field-proven guidance for running a clean, safe, conclusive MDI proof of concept.

Keep the PoC scoped and reversible

- Install **one** sensor on a **non-critical or lab domain controller** (`MDI PoC Sensor – Pilot DC – Pilot`). The whole value is that you can uninstall it cleanly without touching the rest of the fleet.
- Preview every change with `-WhatIf` first (`POCMDI-Enable.ps1`, `POCMDI-Teardown.ps1`).

Benign signal only — never real attacks

- Use **read-only directory enumeration** and a **honeytoken** decoy account exclusively. They exercise the detection pipeline without touching anything.
- **Never** run credential-dumping tools, exploitation frameworks, DCSync, or anything that modifies AD. MDI detects the *pattern* of enumeration from benign commands — you do not need a real attack to prove it. Adversary emulation belongs in a separate, governed engagement.

Use a honeytoken for a clean, high-fidelity signal

- Tag a **dormant, unused** account as the honeytoken (`MDI PoC Honeytoken – Pilot`) — never a real, in-use identity. Because a honeytoken should never be touched, any activity against it is an unambiguous detection, which makes for a crisp PoC demonstration.

Respect the domain controller

- The sensor runs on a DC — treat it as security-critical infrastructure. Install in an approved **change window**, confirm sensor health, and monitor resources. Uninstall cleanly at teardown.

Set the bar before you run

- Agree the **evaluation criteria** in writing first (Requirement 2.7). Feed them into `POCMDI-Scorecard.ps1` so the verdict is objective.

Allow warm-up before judging

- The identity timeline, hunting tables, and UEBA scoring populate over **minutes to hours** after sensor install. Give the pipeline time before scoring a "fail," and use `POCMDI-Validate.ps1` to check when signal has arrived.

Keep response approval-gated

• Any response action stays in an **approval/simulate** posture for the PoC. The decision to enable automated response is a production decision for the SOC/security team.

Tear down completely

• Untag the honeytoken, uninstall the sensor, remove the PoC-only DSA, and resolve the `MDI-PoC-TESTSIGNAL` alerts. Then confirm with `POCMDI-Validate.ps1`. Keep only the scorecard.

Keep licensing and preview facts current

• MDI is license-gated, and sensor OS support and endpoint URLs change. **Verify current availability, sensor support, and the endpoint list in your tenant** before assuming.

Where to go next

Once the PoC proves MDI, natural next steps in the catalogue:

• The **Defender XDR Deployment & Unified Onboarding** solution to deploy sensors fleet-wide and correlate MDI with Endpoint, Office, and Cloud Apps.

• **Active Directory hardening / tiered admin** solutions to reduce the attack surface MDI watches.

• The **Entra ID governance / Conditional Access** solutions to harden the identity layer end to end.

• The **Sentinel SOC/XDR integration** solution to stream MDI into a SIEM for long-term hunting.

8. Deliverables Summary — Proof-of-Concept Kit: Microsoft Defender for Identity

Documentation

Document	Purpose
Executive Summary	What the PoC proves, the boundary, and the effort
1. Solution Overview	What MDI / ITDR is and how a scoped, safe PoC proves it
2. Requirements Guide	PoC-specific prerequisites with Verify / Implement-if-missing
Global Requirements (Defender)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps — install a pilot sensor, generate benign signal, validate, score, tear down
5. Testing Guide	Eight validation tests with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Safe-PoC guidance and where to go next

PowerShell scripts (Microsoft Graph Security + directory enumeration + KQL)

Script	What it does
`POCMDI-Enable.ps1`	`-WhatIf`-gated pilot enable — previews readiness (plan, DCs, DSA), and can tag the honeytoken decoy `MDI PoC Honeytoken – Pilot` (safe preview by default; sensor install is a guided change-window step)
`POCMDI-TestSignal.ps1`	**Benign** test-signal generator — runs harmless read-only directory enumeration and touches the honeytoken (no attack tooling, ever)
`POCMDI-Validate.ps1`	Read-only demo/validate — confirms the reconnaissance and honeytoken alerts, the identity timeline, and advanced-hunting rows
`POCMDI-Scorecard.ps1`	Scores observations against your evaluation criteria and writes `MDI PoC Scorecard – Pilot DC – Validate`
`POCMDI-Teardown.ps1`	`-WhatIf`-gated clean rollback — untags the honeytoken, guides sensor uninstall + DSA removal, resolves test alerts

What the buyer receives

- This solution as a **Word** document and **PDF**.
- All five **PowerShell scripts** (with embedded KQL) in a single ZIP.
- A repeatable process following the model **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — run here as a safe, reversible PoC with response kept approval-gated.

Delivered as a **Complete Solution Package (PDF + Word + scripts)**.

Outcome

A concrete, in-tenant proof that Microsoft Defender for Identity delivers ITDR — a pilot sensor raising real detections from benign reconnaissance and honeytoken activity, visible on the identity timeline with UEBA scoring — scored against your criteria and cleanly torn down.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

POCMDI-Enable.ps1

Purpose: Prepares the MDI proof of concept on a single pilot domain controller. It previews readiness (MDI plan, admin roles, activation state, candidate DCs, DSA, honeytoken account) and, gated behind -WhatIf, can tag the dormant decoy account as a HONEYTOKEN via Microsoft Graph. The sensor install itself is a guided change-window step on the DC (documented in the Implementation Guide) — this script does not silently touch a DC. HOW TO RUN Preview only (changes nothing): powershell -ExecutionPolicy Bypass -File .\POCMDI-Enable.ps1 -WhatIf Tag the honeytoken decoy account: powershell -ExecutionPolicy Bypass -File .\POCMDI-Enable.ps1 -HoneytokenUpn "MDI PoC Honeytoken - Pilot" Requires Microsoft.Graph.Authentication and Security Administrator for the honeytoken tag. EXPECTED RESULTS A readiness summary, guided sensor-install steps, and (when applied) the honeytoken decoy account tagged so any activity against it is a high-fidelity detection. .NOTES White-label. Read-only by default (-WhatIf). Pilot-scoped (one DC). Reverse with POCMDI-Teardown.ps1. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([string]$HoneytokenUpn = 'MDI PoC Honeytoken - Pilot', [string]$SensorLabel = 'MDI PoC Sensor - Pilot DC - Pilot', [string]$DsaName = 'MDI PoC - DSA - Pilot')

How to run: Preview only (changes nothing): powershell -ExecutionPolicy Bypass -File .\POCMDI-Enable.ps1 -WhatIf Tag the honeytoken decoy account: powershell -ExecutionPolicy Bypass -File .\POCMDI-Enable.ps1 -HoneytokenUpn "MDI PoC Honeytoken - Pilot" Requires Microsoft.Graph.Authentication and Security Administrator for the honeytoken tag. EXPECTED RESULTS A readiness summary, guided sensor-install steps, and (when applied) the honeytoken decoy account tagged so any activity against it is a high-fidelity detection. .NOTES White-label. Read-only by default (-WhatIf). Pilot-scoped (one DC). Reverse with POCMDI-Teardown.ps1. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([string]$HoneytokenUpn = 'MDI PoC Honeytoken - Pilot', [string]$SensorLabel = 'MDI PoC Sensor - Pilot DC - Pilot', [string]$DsaName = 'MDI PoC - DSA - Pilot')

POCMDI-Scorecard.ps1

Purpose: Turns the PoC observations into an objective, shareable scorecard. It re-checks the key MDI capabilities (recon/honeytoken alert, enumeration observed, directory activity, logon context) via read-only advanced hunting, maps each to the pass/fail criteria you agreed up front, and writes a JSON + console scorecard. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\POCMDI-Scorecard.ps1 ` -CriteriaPath .\criteria.json -OutFile "MDI PoC Scorecard - Pilot DC - Validate.json" Sample criteria.json: [{ "id":"alert", "label":"Recon/honeytoken alert fired", "capability":"alert", "required":true }, { "id":"query", "label":"Enumeration observed", "capability":"query", "required":true }, { "id":"dir", "label":"Directory activity on timeline","capability":"directory", "required":true }, { "id":"logon", "label":"Logon context captured", "capability":"logon", "required":false }] Requires Microsoft.Graph.Authentication and ThreatHunting.Read.All consent. EXPECTED RESULTS A per-criterion PASS/FAIL table, an overall pass rate, and a saved scorecard file. .NOTES White-label. Read-only. Whether the PoC "passed" and how criteria are weighted is the customer's decision. > [CmdletBinding()] param([string]$CriteriaPath, [string]$OutFile = 'MDI PoC Scorecard - Pilot DC - Validate.json', [int]$LookbackHours = 12)

How to run: powershell -ExecutionPolicy Bypass -File .\POCMDI-Scorecard.ps1 ` -CriteriaPath .\criteria.json -OutFile "MDI PoC Scorecard - Pilot DC - Validate.json" Sample criteria.json: [{ "id":"alert", "label":"Recon/honeytoken alert fired", "capability":"alert", "required":true }, { "id":"query", "label":"Enumeration observed", "capability":"query", "required":true }, { "id":"dir", "label":"Directory activity on timeline","capability":"directory", "required":true }, { "id":"logon", "label":"Logon context captured", "capability":"logon", "required":false }] Requires Microsoft.Graph.Authentication and ThreatHunting.Read.All consent. EXPECTED RESULTS A per-criterion PASS/FAIL table, an overall pass rate, and a saved scorecard file. .NOTES White-label. Read-only. Whether the PoC "passed" and how criteria are weighted is the customer's decision. > [CmdletBinding()] param([string]$CriteriaPath, [string]$OutFile = 'MDI PoC Scorecard - Pilot DC - Validate.json', [int]$LookbackHours = 12)

POCMDI-Teardown.ps1

Purpose: Returns the tenant/AD to its pre-PoC state. It: - reminds you to untag the honeytoken decoy account (Entity tags -> Honeytoken), - prints the exact steps to delete the sensor and uninstall the sensor service on the DC, - reminds you to remove the PoC-only DSA and resolve the MDI-PoC-TESTSIGNAL test alerts. Because sensor uninstall runs ON the domain controller, this script does not silently remove it — it guides the change-window step. Every action is gated behind -WhatIf. HOW TO RUN Preview: powershell -ExecutionPolicy Bypass -File .\POCMDI-Teardown.ps1 -WhatIf Apply (prints the guided steps and marks intent): powershell -ExecutionPolicy Bypass -File .\POCMDI-Teardown.ps1 EXPECTED RESULTS The honeytoken untagged, the sensor removed, the DSA removed (if PoC-only), and test alerts resolved. .NOTES White-label. Reversible teardown. Sensor uninstall is a guided change-window step on the DC. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([string]$SensorLabel = 'MDI PoC Sensor - Pilot DC - Pilot', [string]$HoneytokenUpn = 'MDI PoC Honeytoken - Pilot', [string]$DsaName = 'MDI PoC - DSA - Pilot', [string]$Tag = 'MDI-PoC-TESTSIGNAL')

How to run: Preview: powershell -ExecutionPolicy Bypass -File .\POCMDI-Teardown.ps1 -WhatIf Apply (prints the guided steps and marks intent): powershell -ExecutionPolicy Bypass -File .\POCMDI-Teardown.ps1 EXPECTED RESULTS The honeytoken untagged, the sensor removed, the DSA removed (if PoC-only), and test alerts resolved. .NOTES White-label. Reversible teardown. Sensor uninstall is a guided change-window step on the DC. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([string]$SensorLabel = 'MDI PoC Sensor - Pilot DC - Pilot', [string]$HoneytokenUpn = 'MDI PoC Honeytoken - Pilot', [string]$DsaName = 'MDI PoC - DSA - Pilot', [string]$Tag = 'MDI-PoC-TESTSIGNAL')

POCMDI-TestSignal.ps1

Purpose: Produces harmless, READ-ONLY identity signal so the MDI detection pipeline has something to catch. Run this from a domain-joined TEST WORKSTATION signed in as a TEST ACCOUNT. It: 1. Runs benign directory enumeration (net group / net user / nltest / an LDAP read) — the same commands admins run daily. MDI flags the reconnaissance PATTERN; nothing is changed. 2. Touches the honeytoken decoy account with a benign lookup so that clean, high-fidelity signal is raised. NO attack tooling, credential dumping, exploitation, or AD modification is ever performed. -WhatIf shows what it would run without running it. HOW TO RUN Preview: powershell -ExecutionPolicy Bypass -File .\POCMDI-TestSignal.ps1 -WhatIf Generate benign signal (run on the test workstation as the test account): powershell -ExecutionPolicy Bypass -File .\POCMDI-TestSignal.ps1 -HoneytokenSam "mdi-poc-honeytoken" EXPECTED RESULTS Reconnaissance and honeytoken detections appear in MDI for the PoC to validate. .NOTES White-label. BENIGN, READ-ONLY signal only (enumeration + honeytoken lookup). Never real attacks. > [CmdletBinding(SupportsShouldProcess = $true)] param([string]$HoneytokenSam, [string]$Tag

= 'MDI-PoC-TESTSIGNAL')

How to run: Preview: powershell -ExecutionPolicy Bypass -File .\POCMDI-TestSignal.ps1 -WhatIf Generate benign signal (run on the test workstation as the test account): powershell -ExecutionPolicy Bypass -File .\POCMDI-TestSignal.ps1 -HoneytokenSam "mdi-poc-honeytoken" EXPECTED RESULTS Reconnaissance and honeytoken detections appear in MDI for the PoC to validate. .NOTES White-label. BENIGN, READ-ONLY signal only (enumeration + honeytoken lookup). Never real attacks. > [CmdletBinding(SupportsShouldProcess = $true)] param([string]$HoneytokenSam, [string]$Tag = 'MDI-PoC-TESTSIGNAL')

POCMDI-Validate.ps1

Purpose: Read-only validation and live-demo helper for the MDI proof of concept. Using the Microsoft 365 Defender advanced-hunting API (via Microsoft Graph Security), it confirms the benign test signal surfaced as: - AlertInfo — a reconnaissance / honeytoken alert reached the portal - IdentityQueryEvents — the enumeration was observed - IdentityDirectoryEvents — directory activity on the identity timeline - IdentityLogonEvents — logon/authentication context It also points to the identity timeline / UEBA views for the demo. Makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\POCMDI-Validate.ps1 Requires Microsoft.Graph.Authentication and ThreatHunting.Read.All consent. EXPECTED RESULTS A pass/fail line per capability, plus portal paths for the identity timeline / UEBA demo. .NOTES White-label. Read-only. Advanced-hunting queries only read telemetry; they change nothing. > [CmdletBinding()] param([int]$LookbackHours = 6)

How to run: powershell -ExecutionPolicy Bypass -File .\POCMDI-Validate.ps1 Requires Microsoft.Graph.Authentication and ThreatHunting.Read.All consent. EXPECTED RESULTS A pass/fail line per capability, plus portal paths for the identity timeline / UEBA demo. .NOTES White-label. Read-only. Advanced-hunting queries only read telemetry; they change nothing. > [CmdletBinding()] param([int]$LookbackHours = 6)